

## **Healthcare Resource Availability: Analyzing Global Disparities**

### **ADAN8880 - Directed Practicum in Analytics**

**Professor: Nurtekin Savas**

**Date: March 23, 2025**

**Group Members: Jen Jiang, Yiwen Ma**

## **Week 8 Written Report**

### **Modeling Approach**

For this week's task, we chose LightGBM as our modeling approach. It's a gradient boosting framework that's known for being fast, efficient, and highly effective on structured data, which is ideal for estimating our target variable of life expectancy using socio-economic and other health indicators. Our dataset includes a wide range of variables with complex and nonlinear relationships, and LightGBM is well-equipped to handle those patterns through its ability to model interactions and capture subtle trends.

Another advantage of LightGBM is that it handles missing values automatically and includes built-in regularization to help prevent overfitting. That said, its flexibility and complexity also mean it requires careful hyperparameter tuning to get the best performance. Overall, LightGBM offered the right balance of power and adaptability for the problem we were solving this week.

### **Hyperparameters**

To evaluate model performance, we defined and tested three variations of LightGBM models with key hyperparameter adjustments:

1. LGBM default: base model with minimal tuning.
2. LGBM feature subsampling: adjusted `learning_rate`, `max_depth`, `subsample`, and `colsample_bytree` to reduce overfitting and introduce model robustness through random sampling.
3. LGBM regularized: introduced stronger regularization by modifying `reg_alpha` and `reg_lambda`, along with a lower `learning_rate` and shallower trees to control model complexity.

These hyperparameters were selected because they have a direct impact on model generalization. Learning rate and number of estimators affect how fast and deeply the model learns. Subsampling and `colsample_bytree` help reduce variance, and regularization parameters such as L1 and L2 penalties help prevent overfitting by constraining model complexity.

### **Evaluation Metrics**

We evaluated each model using RMSE and R-squared for both training and validation datasets. RMSE was chosen because it penalizes large prediction errors and is easy to interpret in the units of the target variable. R-squared was included to assess how well the model explains the variability in the data. Using both metrics together provides a more complete picture of model performance.

### Training and Validation Results

Below are the results for the three model variations, along with the best model selected by GridSearchCV:

| Model                    | Train RMSE | Validation RMSE | Train R-squared | Validation R-squared |
|--------------------------|------------|-----------------|-----------------|----------------------|
| LGBM default             | 0.2259     | 0.0976          | 0.9994          | 0.9999               |
| LGBM feature subsampling | 0.4391     | 0.2122          | 0.9978          | 0.9993               |
| LGBM regularized         | 3.1191     | 2.5652          | 0.8886          | 0.9021               |
| Best LGBM via GridSearch | 0.3564     | 0.1768          | 0.9985          | 0.9995               |

- Among the three manually defined variations, the LGBM default model had the lowest RMSE and highest R-squared across both training and validation sets, which demonstrates strong predictive performance with minimal tuning.
- The feature subsampling model also performed well, maintaining strong validation metrics and also adding slight robustness through subsampling.
- In contrast, the LGBM regularized model significantly underperformed, with higher RMSE values. This is likely due to overly aggressive regularization and conservative learning settings, such as a very low learning rate and shallow trees, which led to underfitting. While regularization is useful to prevent overfitting, these results show that excessive constraints can suppress the model's ability to learn meaningful patterns.
- The GridSearchCV model found a strong hyperparameter configuration, but it did not outperform the default. It showed slightly higher validation RMSE and slightly lower R-squared compared to the LGBM Default.

Running these three LightGBM variations helped us understand how different hyperparameter configurations affect model complexity, learning behavior, and generalization. These manual tests provided a foundation for comparison before moving on to automated tuning.

After evaluating the manual models, we applied GridSearchCV to automatically search a wider range of hyperparameter combinations. We used its default scoring method of minimizing RMSE to systematically identify the best-performing configuration from the grid.

While the GridSearchCV model performed well, the manually defined LGBM default model still outperformed all others on the validation set, with the lowest RMSE and highest R-squared. Therefore, we selected the LGBM default as the final model for this week. It demonstrated strong generalization and accuracy with minimal complexity, which makes it the most reliable choice for our life expectancy forecasting task.

### Test Set Performance

To further validate generalization, we evaluated both the LGBM Default model and the GridSearchCV-selected model on the unseen test dataset:

| Model              | Train RMSE | Test RMSE | Train R-squared | Test R-squared |
|--------------------|------------|-----------|-----------------|----------------|
| LGBM default       | 0.2259     | 0.4244    | 0.9994          | 0.9973         |
| GridSearchCV Model | 1.2138     | 1.1582    | 0.9831          | 0.9799         |

- The results confirm that the LGBM default model not only performed best on the validation set but also maintained superior accuracy on the test set. It achieved the lowest RMSE and highest R-squared across both datasets, showing consistent performance and strong generalization.
- Even though the GridSearchCV model remains a strong alternative, its test metrics were slightly weaker. This shows that the simpler and manually configured default model was better suited for this prediction task as it balances performance with efficiency. Therefore, the LGBM default remains our selected final model for this week.

#### Cross-Validation Summary

To evaluate the stability of the GridSearchCV-selected model, we ran 5-fold cross-validation using RMSE as the scoring metric. Initially, we observed high variance in RMSE across folds, with one fold showing a significantly higher error. This was likely due to the default KFold splitting the data in order, which may have created unbalanced folds. To address this, we updated our approach by enabling `shuffle=True` in the KFold object. This ensured the data was randomly distributed across folds, which resulted in more balanced and representative splits.

After shuffling, the cross-validation results became much more stable, with an average RMSE of 1.09 and a lower standard deviation of 0.27. This indicates that the GridSearchCV model performs consistently across different subsets of the training data.

While the cross-validation results confirm that the GridSearchCV model is generally stable and reliable, the LGBM default model still delivered stronger performance on both the validation and test sets. Given its simplicity, consistency, and accuracy, we're confident in selecting the LGBM Default as the final model for this week's forecasting task.